

IdeaLaunch PRD

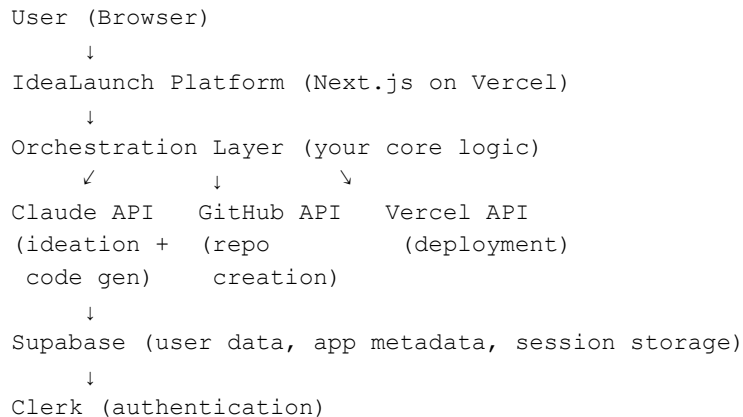
Product Requirements Document

10. MVP Technical Architecture (Deep Dive)

Architecture Philosophy

The MVP should be entirely "wired together" from existing services — you are not building infrastructure, you are building the orchestration layer that connects best-in-class tools through their APIs. Think of it as a conductor, not a musician.

System Architecture Overview



Phase-by-Phase Technical Breakdown

Phase 1 — Idea Intake

- Simple conversational UI built with Next.js + shadcn/ui components
- Claude API handles the conversation, asking structured clarifying questions
- You define a system prompt that extracts: app purpose, target user, core features, tone, and constraints
- Output is a structured JSON object stored in Supabase

Phase 2 — Requirements Generation

- Claude takes the structured JSON and generates a plain-language requirements summary
- User reviews and approves via a simple checklist UI
- Approved requirements stored in Supabase and passed to the build phase
- This is your moat — no competitor does this step for non-technical users

Phase 3 — Code Generation

- Claude (or GPT-4o, worth testing both) receives the approved requirements and generates a complete Next.js application
- Code is generated in a structured format: file tree + file contents
- You parse the output and write files programmatically
- Use E2B (e2b.dev) as a sandboxed code execution environment to preview and validate before deployment
- E2B gives you a live browser preview inside your platform — this is the "magic moment" for the user

Phase 4 — GitHub Repo Creation

- GitHub API creates a new private repo under a service account
- Your orchestration layer commits the generated files to the repo automatically
- User optionally gets access to the repo if they want it (advanced toggle)

Phase 5 — Vercel Deployment

- Vercel API deploys directly from the GitHub repo
- Environment variables (Supabase keys, OpenAI keys, etc.) are injected automatically
- Vercel returns a live deployment URL
- URL is stored in Supabase and displayed in the user dashboard

Key Technical Decisions for MVP

LLM Choice: Start with Claude API (Anthropic) for both ideation and code generation. Claude is strong at following structured output instructions which is critical for generating parseable file trees. Have GPT-4o as a fallback and test both on code quality.

Code Sandbox: E2B is purpose-built for this use case — it spins up a sandboxed Next.js environment in seconds and gives you a live preview URL. This is essential for the "see your app before it goes live" moment.

Database: Supabase handles user accounts, app metadata, conversation history, and generated file storage. Free tier is sufficient for MVP.

Deployment Target: Vercel only for MVP. It has the best programmatic deployment API, the fastest cold starts, and free tier is generous. Custom domains, AWS, and other targets are v2.

Auth: Clerk is the fastest to implement and has the best UI out of the box. Integrate in a day, move on.

What You Are Actually Building (vs. Buying)

Component	Build or Buy
Conversational intake UI	Build (simple chat UI in Next.js)
Orchestration logic	Build (this is your core IP)
LLM calls	Buy (Claude/OpenAI API)
Code sandbox/preview	Buy (E2B)
Repo management	Buy (GitHub API)
Deployment	Buy (Vercel API)

Component	Build or Buy
Auth	Buy (Clerk)
Database	Buy (Supabase)
Payments	Buy (Stripe)

Your real build surface is small: a conversational UI, the orchestration logic that strings these services together, and the prompts that make the output high quality. The prompts are actually where most of your iteration will happen early on.

Estimated Build Timeline to Working MVP

- Week 1–2: Auth, basic UI shell, Claude API intake conversation working
- Week 3–4: Requirements generation + approval flow, E2B preview working
- Week 5–6: GitHub + Vercel deployment pipeline connected end-to-end
- Week 7–8: Polish, error handling, user dashboard, first real user tests

A solo developer or a small team could have a functional end-to-end demo in 6–8 weeks.

11. Validation User Profile (Deep Dive)

Who You're Looking For in Early Testers

The goal of early validation is not to find people who are impressed by the tech — it's to find people who are genuinely stuck between an idea and a live product. Those people exist in abundance and they will give you honest, motivated feedback.

The Ideal Early Validation User

- Has had an app or tool idea for more than 3 months but hasn't acted on it
- Has tried and failed with existing tools (Bubble, Webflow, Glide) or gave up after hitting a wall
- Is not a developer and has no plans to hire one yet
- Has some willingness to pay — they see the idea as a real business opportunity, not a hobby

Red Flags in Early Testers to Avoid

- Developers or technical people — they will critique the code, not the experience
- People who "just want to explore" with no real idea in mind
- Anyone who says "I could build this myself" — they are not your user

Validation Questions to Ask Before the Session

1. Can you describe your app idea in one sentence?
2. What have you tried so far to build it or get it built?
3. What stopped you?
4. If this existed today, would you pay for it? What would feel fair?

What Success Looks Like in Validation

- User completes the intake flow without asking for help
- User says "oh wow" when they see the live preview — that emotional reaction is your signal
- User asks "how do I share this?" or "can I add X feature?" — forward-looking questions mean they believe in the output
- At least 3 out of 5 users say they would pay for this

Validation Format

Run each session as a 45-minute structured test. First 10 minutes is conversation about their idea, next 25 minutes they use the platform (you watch and say nothing), last 10 minutes is debrief. Record everything with permission. You will learn more in 5 sessions than in 5 weeks of building.

12. Leveraging Your Network

You have an unusually strong network for this kind of product — people in product, engineering, design, content, and brand. Here's how to deploy each group strategically rather than just asking for generic feedback.

Product People (Big Tech)

These are your most valuable early advisors. They think in frameworks, user journeys, and prioritization — exactly what you need right now.

How to engage them: Don't ask for generic feedback. Come with specific questions. Ask them to pressure-test your user journey, poke holes in your MVP scope, and tell you what you're missing from a product strategy standpoint. A 30-minute call with a senior PM who has launched a developer tool is worth weeks of solo thinking.

Specific ask: *"I have a PRD draft — would you spend 30 minutes tearing it apart with me?"*

Architects & Developers

These folks will save you from making expensive technical decisions early that you'll regret later. They are not your end users, but they are your reality check.

How to engage them: Share the technical architecture section specifically. Ask: "Is there anything here that will break at scale or that I'll regret at 1,000 users?" Also ask them to recommend the best person they know who fits your ideal validation user profile — developers often know non-technical entrepreneurs who have come to them for help building something.

Specific ask: *"Does this architecture make sense for an MVP? What would you change?"*

Designers & UX People

The user experience of the intake flow is everything. If the first conversation feels clunky or robotic, non-technical users will bounce before they ever see the magic.

How to engage them: Ask them to review your user journey map and the conversational intake flow specifically. Better yet, ask one or two of them to do a low-fidelity wireframe of what the ideal onboarding experience looks like for a nervous, non-technical user.

Specific ask: *"Can you wireframe what the first 3 screens of this should feel like for someone who has never built an app?"*

Content Strategists

The language of your platform is a product decision. Non-technical users are scared of jargon. Every label, prompt, error message, and button needs to feel approachable and confidence-building.

How to engage them: Give them the intake conversation flow and ask them to rewrite it as if they were explaining it to a smart friend with no tech background. Also ask them to help you name things — "requirements" should probably be called something warmer, like "your app blueprint" or "your idea summary."

Specific ask: *"Can you audit the language in this flow and flag anything that would feel intimidating to a non-technical user?"*

Brand Strategists

It's early for deep brand work, but a brand strategist can help you nail the positioning statement and the emotional promise of the product before you build a landing page or start talking to investors.

How to engage them: Share the vision and competitive analysis. Ask them to help you articulate what makes IdeaLaunch different in one sentence — not a feature comparison, but an emotional positioning. "Bolt.new builds apps. IdeaLaunch launches ideas." That kind of clarity.

Specific ask: *"Help me find the one sentence that makes someone feel like this was built for them."*

How to Organize Your Network Outreach

Rather than doing this ad hoc, run it like a lightweight advisory process. Send a personalized note to 2-3 people in each category with a specific ask and a time estimate (most asks should be 30 minutes or less). Keep a simple spreadsheet tracking who you've talked to, what they said, and what you changed because of it. This also becomes your early community — people who helped shape the product are your first evangelists.